Informe del proyecto de automatización — Demo Web Shop

ISW-622 Pruebas de Software · III Cuatrimestre 2026 · UTN Sede San Carlos

Daniela Rojas · Brandon Prado

3 de noviembre de 2026

Docente: Jose David Carvajal Jiménez · **Sitio bajo prueba:** https://demowebshop.tricentis.com · **Repositorio:** https://github.com/UTN-Estudiantes/proyecto1/tree/DanielaRojas-BrandonPrado

1. Estrategia: qué automatizamos y por qué

Diseñamos **45 casos de prueba** escritos para los 11 requerimientos (RF-01 a RF-11): 25 positivos, 15 negativos y 5 de límite (44 % negativos o de límite). Automatizamos **41** con JUnit 5 y Page Object Model sobre **14 clases de página** con Page Factory. Las pruebas cubren los 11 requerimientos, incluyen 20 casos negativos o de límite y el flujo completo de compra (CP-034).

Criterio para automatizar. Le dimos prioridad a lo que (1) tiene prioridad Alta en los requerimientos (registro, sesión, búsqueda, carrito, productos con opciones y compra), (2) se repite en cada versión del sitio y conviene tener como prueba de regresión, y (3) tiene un resultado verificable sin intervención humana (un mensaje, un contador, un precio, una URL).

Qué no automatizamos y por qué.

- **CP-027 cantidad no numérica o negativa:** el sitio no muestra un mensaje documentado; primero hay que definir el comportamiento esperado con el negocio, así que se dejó como caso manual y se reporta como hallazgo.
- **CP-037 compra como invitado, CP-044 historial de pedidos, CP-045 direcciones:** son automatizables reutilizando `CheckoutPage` y `CuentaPage`, pero los dejamos para una siguiente iteración: el flujo de compra ya está cubierto por CP-034 y cada compra extra alarga la ejecución en un sitio compartido.
- No hicimos pruebas de carga ni de seguridad, por las reglas de uso del sitio.

Independencia. Cada prueba arranca con un navegador limpio (`BaseTest`) y, si necesita un usuario, registra uno nuevo con un correo único (hora en milisegundos más un contador). Así ninguna prueba depende del orden ni de datos que dejó otra ejecución.

2. Retos de automatización y cómo los resolvimos

Reto	Solución
Datos únicos en el registro	`DatosPrueba.correoUnico()` genera `qa.utn.<ms>.<n>@prueba.com`; el contador evita choques si dos registros caen en el mismo milisegundo.
Localizadores con parámetro (botón *Add to cart* de un producto, categoría del menú, etiqueta de una opción)	`@FindBy` no acepta parámetros, así que dentro de la página usamos `driver.findElement(By...)` con un XPath relativo basado en el nombre del producto, y lo justificamos con un comentario. Las pruebas nunca usan `By`.
Opciones obligatorias y listas (procesador, RAM, disco, país, estado)	`Select` para las listas; para los botones de opción buscamos la etiqueta `label[for=id]` de cada radio y marcamos la que contiene el texto pedido.
Mensajes dinámicos (barra verde/roja al agregar al carrito, aviso del boletín)	Esperamos con `WebDriverWait` a que el aviso sea visible y tenga texto, lo guardamos y cerramos la barra, porque tapa el encabezado y bloquea clics posteriores.
Compra de varios pasos (6 pasos cargados por AJAX)	Antes de cada paso esperamos a que su botón *Continue* sea clickeable; los localizadores empiezan con el id del contenedor del paso (`#shipping-method-buttons-container ...`) porque varios botones comparten clase. Al confirmar se espera la URL `/checkout/completed`.
Listas que recargan la página (orden, productos por página, vista)	Después de elegir la opción esperamos a que la URL contenga el parámetro (`orderby=`, `pagesize=`) y devolvemos una página nueva para evitar elementos obsoletos.
Estado del carrito y de la sesión	Navegador nuevo por prueba; nunca reutilizamos un usuario.

No usamos `Thread.sleep`: la única pausa es `Visual.pausa()` del proyecto base, solo para observar, y se apaga con `-Dritmo=0` sin que las pruebas fallen.

3. Hallazgos

> Cada hallazgo se confirmó a mano en el sitio el 3 de noviembre de 2026. (Borrar esta nota al final.)

H-01 · Cantidad 0 elimina el producto sin aviso (RF-05). El requerimiento dice que "se verifican cantidades no válidas", pero con cantidad 0 el sitio quita el producto en silencio. *Pasos:* 1) agregar *14.1-inch Laptop* al carrito; 2) escribir 0 en *Qty*; 3) *Update shopping cart*. *Real:* carrito vacío sin mensaje. *Esperado:* aviso de cantidad no válida o confirmación de eliminación.

H-02 · Cantidad negativa o no numérica (RF-05). *Pasos:* igual que H-01 con −1 y con `abc`. *Real:* (anotar lo observado). *Esperado:* mensaje de cantidad no válida sin cambiar el carrito.

H-03 · La encuesta exige estar registrado (RF-11). El requerimiento dice "se puede votar en la encuesta", sin restricción. *Pasos:* como visitante, elegir una respuesta en *Community poll* y pulsar *Vote*. *Real:* "Only registered users can vote." Es un desajuste entre requerimiento y sitio, o un requerimiento incompleto.

H-04 · Mínimo de contraseña no especificado (RF-01). El requerimiento solo dice "un mínimo de caracteres". El sitio exige 6 (con 5 muestra "The password should have at least 6 characters."). Recomendamos que el requerimiento diga el valor exacto.

H-05 · Mensaje del login con campos vacíos (RF-02). *Pasos: Log in* sin escribir nada. *Real:* (anotar el mensaje exacto). Si el mensaje es genérico ("No customer account found") en vez de indicar que el correo es obligatorio, se reporta como mejora de usabilidad.

H-06 · Datos compartidos del sitio. Al ser público, el catálogo y los precios pueden cambiar; por eso las pruebas de búsqueda y orden comprueban reglas (orden ascendente, rango de precio) en lugar de listas fijas de productos.

4. Aprendizajes y mejoras posibles

- Separar la página (cómo se encuentra cada elemento) de la prueba (qué se espera) hizo que, cuando un localizador cambió, solo tocáramos una clase.
- Las esperas explícitas son lo que vuelve estable una prueba de un sitio con AJAX; una pausa fija solo esconde el problema.
- Escribir primero los casos y la trazabilidad nos hizo ver que RF-06 a RF-08 tenían pocos casos negativos.
- **Mejoras:** automatizar CP-037, CP-044 y CP-045; parametrizar casos con `@ParameterizedTest` (por ejemplo, varios correos inválidos); agregar capturas automáticas cuando una prueba falla; ejecutar las pruebas en paralelo en un servidor de integración continua.